Task #1: Basic Queue Operations

Problem: Implement a standard **Linear Queue** using an integer array of size $N = 5$. You do not need to handle circular wrapping.
Requirement: Write a program that processes a sequence of commands:

- Command '1 X': Enqueue integer X. Print "Overflow" if full.
- Command '2': Dequeue an element. Print "Underflow" if empty.
- Command '3': Display current queue.
- Command '0': Exit.

Input Format: Lines containing integers representing commands.
Sample Input:

```
1 10
1 20
1 30
2
3
0
```

Sample Output:

```
Enqueued 10
Enqueued 20
Enqueued 30
Dequeued 10
Queue: 20 30
```

Task #2: Depth First Search (DFS)

Problem: Given a graph with V vertices and E edges, perform a **Depth First Search (DFS)** traversal starting from **Node 0**. Always visit neighbors in ascending order (smallest node first).
Input Format:

- First line: Two integers V and E.
- Next E lines: Two integers u v representing an edge between u and v.

Sample Input:

```
5 4
0 1
0 2
1 3
1 4
```

Sample Output:

```
DFS Traversal: 0 1 3 4 2
```

Task #3: The "Mirror" Binary Search Tree

Problem: Implement a **Modified BST** that supports both **Insertion** and **Deletion**. The tree must maintain the following property:

- **Left Child:** Contains values **greater than** the parent.
- **Right Child:** Contains values **less than or equal to** the parent.

Requirement:

- Implement `insert(int key)`: Add a value maintaining the mirror property.
- Implement `delete(int key)`: Remove a value. If the node has two children, maintain the structure correctly. (Your tree structure may differ from the sample)
- Implement `inOrder()`: Print the tree when commanded.

Input Format:

- First line: Integer Q (Number of queries).
- Next Q lines: '1 X' (Insert), '2 X' (Delete), or '3' (Print In-order).

Sample Input:

```
7
1 50
1 70
1 30
1 40
2 30
2 50
3
```

Sample Output:

```
In-Order: 70 40
```

(Explanation: Insert 50, 70(L), 30(R), 40(L of 30). Delete 30. Delete 50 (Root). Remaining: 70, 40. Command 3 prints result.)

Task #4: Network Vulnerability Analysis

Scenario: You are analyzing a network of bunkers (nodes) and secure lines (edges). A "Critical Point" is a bunker whose removal disconnects the remaining network. Find all such Critical Points (Articulation Points).

Input Format:

- First line: V and E.
- Next E lines: Connections between bunkers (u, v).

Sample Input:

```
5 5
1 0
0 2
2 1
0 3
3 4
```

Sample Output:

```
Critical Points: 0 3
```

1. Basic Structure & I/O

```
#include <iostream>
#include <vector>
#include <algorithm> // for min, max, sort
#include <stack>
#include <queue>
using namespace std;
int main() {
    int x;
    cin >> x;            // Input
    cout << x << endl;   // Output with
   newline
    return 0;
}
```

2. Arrays & Pointers

```
// Fixed Size Array
int arr[100];
arr[0] = 5;
// Dynamic Allocation
int* ptr = new int(10); // Single int
int* arrPtr = new int[50]; // Array
// Structure Pointer Syntax
struct Item {
    int id;
    Item* link;
};
// Accessing members via pointer
Item* obj = new Item();
obj->id = 10;       // Arrow operator
obj->link = NULL;   // Null pointer
```

3. STL Vector (std::vector)

```
vector<int> v;

// Adding elements
v.push_back(10);
v.push_back(20);
// Accessing elements
int val = v[0];
int last = v.back();
// Size
int len = v.size();
// Array of Vectors (2D structure)
vector<int> group[100];
group[0].push_back(5); // Add 5 to list 0
```

4. Iteration Syntaxes

```
// Standard For Loop
for (int i = 0; i < 10; i++) {
    // code
}
// Range-based For Loop (C++11)
// Useful for iterating vectors
vector<int> nums = {1, 2, 3};
for (int x : nums) {
    cout << x << ‘‘ ‘‘;
}
```

5. STL Containers

Stack (LIFO):

```
stack<int> s;
s.push(1);     // Insert
s.pop();       // Remove top
int t = s.top(); // Peek top
if (s.empty()) { /* check empty */ }
```

Queue (FIFO):

```
queue<int> q;
q.push(1);       // Insert
q.pop();         // Remove front
int f = q.front(); // Peek front
if (q.empty()) { /* check empty */ }
```

6. Common Utilities

```
// Min / Max
int a = 5, b = 10;
int small = min(a, b);
int big = max(a, b);
// Sorting a Vector
vector<int> data = {5, 1, 3};
sort(data.begin(), data.end());
// data is now {1, 3, 5}
// Swapping variables
swap(a, b);
```

7. Struct Definition Template

```
struct MyData {
    int val;
    // Pointers to other MyData objects
    MyData *next, *prev;
    // Constructor (Optional)
    MyData(int v) {
        val = v;
        next = NULL;
        prev = NULL;
    }
};
```